Optymalizacja receptury batona ALGO-BAR

Algorytm Harmony Search — opis matematyczny

Contents

1 Definicja problemu **2**
1.1 Parametry surowcow 2

2 Sformulowanie matematyczne **2**
2.1 Funkcja celu 2
2.2 Ograniczenia 2
2.3 Pelna postac problemu optymalizacji 3

3 Algorytm Harmony Search **3**
3.1 Parametry algorytmu 3
3.2 Pseudokod algorytmu 3
3.3 Schemat decyzyjny improwizacji 3

4 Funkcja celu z kara (penalty method) **3**

5 Projekcja na ograniczenie rownosci **4**

6 Wyniki optymalizacji **4**

1 Definicja problemu

Celem jest wyznaczenie wektora mas skladnikow:

$$\mathbf{x} = (x_1,\ x_2,\ x_3,\ x_4) \in \mathbb{R}^4$$

minimalizujacego koszt surowcow na 100 g batona, przy zachowaniu ograniczen technologicznych i dietetycznych.

1.1 Parametry surowcow

Skladnik	Symbol	Bialko [%]	Tluszcz [%]	Cena [PLN/100g]	Zakres [g]
Izolat serwatki	x_1	90	0	18,00	[10; 40]
Pasta orzechowa	x_2	25	50	6,00	[15; 40]
Syrop ryzowy	x_3	0	0	2,50	[15; 35]
Ekspandowana quinoa	x_4	14	6	5,50	[5; 25]

Table 1: Dane surowcow batona ALGO-BAR

2 Sformulowanie matematyczne

2.1 Funkcja celu

Minimalizowany jest calkowity koszt receptury [PLN/100 g]:

$$\min_{\mathbf{x}}\ f(\mathbf{x}) \ = \ \frac{1}{100}\sum_{i=1}^{4} c_i\, x_i \ = \ \frac{1}{100}\big(18{,}00\, x_1 + 6{,}00\, x_2 + 2{,}50\, x_3 + 5{,}50\, x_4\big)$$

2.2 Ograniczenia

Zakresy mas:

$$l_i \ \le \ x_i \ \le \ u_i, \qquad i = 1, 2, 3, 4$$

$$10 \le x_1 \le 40, \quad 15 \le x_2 \le 40, \quad 15 \le x_3 \le 35, \quad 5 \le x_4 \le 25$$

Suma mas:

$$\sum_{i=1}^{4} x_i \ = \ 100 \text{ g}$$

Minimalna zawartosc bialka:

$$P(\mathbf{x}) \ = \ 0{,}90\, x_1 + 0{,}25\, x_2 + 0{,}14\, x_4 \ \ge \ 20 \text{ g}$$

Maksymalna zawartosc tluszczu:

$$T(\mathbf{x}) \ = \ 0{,}50\, x_2 + 0{,}06\, x_4 \ \le \ 20 \text{ g}$$

2.3 Pelna postac problemu optymalizacji

$$\boxed{\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) = \frac{1}{100}\sum_{i=1}^{4} c_i\, x_i \\ \text{przy:} \quad & \sum_{i=1}^{4} x_i = 100 \\ & P(\mathbf{x}) \geq 20 \\ & T(\mathbf{x}) \leq 20 \\ & l_i \leq x_i \leq u_i, \quad i = 1,2,3,4 \end{aligned}}$$

3 Algorytm Harmony Search

3.1 Parametry algorytmu

Symbol	Nazwa	Wartosc	Opis
HMS	Rozmiar pamieci harmonii	20	Liczba przechowywanych rozwiazan
HMCR	Wsp. uwzglednienia pamieci	0,90	P(losuj z pamieci)
PAR	Wsp. korekty dzwieku	0,30	P(perturbuj wartosc)
BW	Szerokosc pasma	0,05	Maks. korekta [% zakresu]
NI	Liczba improwizacji	10 000	Liczba iteracji algorytmu

Table 2: Parametry algorytmu Harmony Search

3.2 Pseudokod algorytmu

3.3 Schemat decyzyjny improwizacji

Dla kazdej zmiennej x_i w kazdej iteracji:

$$x_i^{\text{new}} = \begin{cases} \text{HM}[j][i] + \varepsilon & \text{z prawdopodobienstwem HMCR} \cdot \text{PAR} \\ \text{HM}[j][i] & \text{z prawdopodobienstwem HMCR} \cdot (1 - \text{PAR}) \\ \mathcal{U}(l_i,\, u_i) & \text{z prawdopodobienstwem } 1 - \text{HMCR} \end{cases}$$

4 Funkcja celu z kara (penalty method)

Dla rozwiazan nie spelniajacych ograniczen stosowana jest metoda kary:

$$\tilde{f}(\mathbf{x}) \;=\; f(\mathbf{x}) \;+\; \alpha_1 \max\bigl(0,\;\; 20 - P(\mathbf{x})\bigr) \;+\; \alpha_2 \max\bigl(0,\;\; T(\mathbf{x}) - 20\bigr) \;+\; \alpha_3 \left| \sum_{i=1}^{4} x_i - 100 \right|$$

gdzie $\alpha_1 = \alpha_2 = 0{,}5$ oraz $\alpha_3 = 0{,}1$.

5 Projekcja na ograniczenie rownosci

Poniewaz $\sum x_i = 100$ jest ograniczeniem rownosci, stosowana jest projekcja trzystopniowa:

$$x_3 \leftarrow \text{clip}(100 - x_1 - x_2 - x_4,\ \ l_3,\ \ u_3)$$

$$r \leftarrow 100 - (x_1 + x_2 + x_3 + x_4), \qquad x_1 \leftarrow \text{clip}(x_1 + r,\ \ l_1,\ \ u_1)$$

$$r \leftarrow 100 - (x_1 + x_2 + x_3 + x_4), \qquad x_2 \leftarrow \text{clip}(x_2 + r,\ \ l_2,\ \ u_2)$$

gdzie $\text{clip}(v, a, b) = \max(a, \min(b, v))$.

6 Wyniki optymalizacji

Skladnik	x_i^* [g]	Bialko [g]	Tluszcz [g]	Koszt [PLN]
Izolat serwatki	10,00	9,00	0,00	1,8000
Pasta orzechowa	30,01	7,50	15,01	1,8006
Syrop ryzowy	35,00	0,00	0,00	0,8750
Ekspandowana quinoa	24,99	3,50	1,50	1,3745
RAZEM	**100,00**	**20,00**	**16,51**	**5,8501**

Table 3: Optymalna receptura — Harmony Search (NI = 10 000)

Weryfikacja ograniczen:

$$\sum_{i=1}^{4} x_i^* = 100{,}00 \text{ g} \quad \checkmark$$

$$P(\mathbf{x}^*) = 0{,}90 \cdot 10 + 0{,}25 \cdot 30{,}01 + 0{,}14 \cdot 24{,}99 = 20{,}00 \text{ g} \ \geq\ 20 \text{ g} \quad \checkmark$$

$$T(\mathbf{x}^*) = 0{,}50 \cdot 30{,}01 + 0{,}06 \cdot 24{,}99 = 16{,}51 \text{ g} \ \leq\ 20 \text{ g} \quad \checkmark$$

$$\boxed{f(\mathbf{x}^*) = 5{,}8501 \text{ PLN/100g}}$$

```python
# Krok 1: Inicjalizacja pamieci harmonii
PH      = [losowa_poprawna() for _ in range(HMS)]
KosztyH = [cel(x) for x in PH]

# Kroki 2-4: Petla improwizacji
for it in range(NI):

    # Krok 2: Improwizacja nowej harmonii
    nowe_x = np.empty(4)
    for i in range(4):
        if rng.random() < HMCR:
            # Losuj z pamieci harmonii
            val = PH[rng.integers(0, HMS)][i]
            # Korekta dzwieku (pitch adjustment)
            if rng.random() < PAR:
                val += rng.uniform(-BW, BW) * (GG[i] - DG[i])
        else:
            # Losowa improwizacja z pelnego zakresu
            val = rng.uniform(DG[i], GG[i])
        nowe_x[i] = np.clip(val, DG[i], GG[i])

    # Projekcja: wyznacz x3 tak aby suma = 100 g
    x3 = MASA_BATONA - nowe_x[0] - nowe_x[1] - nowe_x[3]
    nowe_x[2] = np.clip(x3, DG[2], GG[2])
    # Korekcja jesli x3 byl przyciety
    r = MASA_BATONA - nowe_x.sum()
    if abs(r) > 0.01:
        nowe_x[0] = np.clip(nowe_x[0] + r, DG[0], GG[0])
    r = MASA_BATONA - nowe_x.sum()
    if abs(r) > 0.01:
        nowe_x[1] = np.clip(nowe_x[1] + r, DG[1], GG[1])

    # Krok 3: Aktualizacja pamieci harmonii
    najgorszy = int(np.argmax(KosztyH))
    nowy_koszt = cel(nowe_x)
    if nowy_koszt < KosztyH[najgorszy]:
        PH[najgorszy]      = nowe_x
        KosztyH[najgorszy] = nowy_koszt

# Krok 5: Najlepsze rozwiazanie
najlepszy = int(np.argmin(KosztyH))
x_opt     = PH[najlepszy]
```

Listing 1: Harmony Search – implementacja Python (hs_algobar.py)